



STAMPED Compliance Checklist

Assess your research objects against the STAMPED principles

TOTAL



7✓

23✗

0?

23%

MUST



5✓

12✗

0?

29%

SHOULD



2✓

6✗

0?

25%

MAY



0✓

5✗

0?

0%

This checklist helps you assess compliance with the **STAMPED** principles for computational reproducibility. Answer each item with Yes/No, and include a reason when needed. Your progress is tracked above, and you can print your results at any time.

MUST — Required

SHOULD — Recommended

MAY — Optional

Are all files and directories nested under a common root?

✓ Yes

✗ No

Are datasets included, linked, or referenced reachable from the code and environment specifications without crossing the boundary of a common root?

✓ Yes

✗ No

Is external software included in the environment or linked as submodules within the project boundary?

✓ Yes

✗ No

Reason for not meeting requirement:

0/250

MUST **T.1 + T.3 + T.5** Tracking



2/3

Persistent content identification **MUST** be recorded for all components. Provenance of all modifications **MUST** be recorded. Provenance records **MUST** include the versions of all components involved.

Are version control systems such as **Git** used for code, text, documentation, and configuration files?

✓ Yes

✗ No

Are version control systems such as **git-annex**, DataLad, or **Git** LFS used for large binary data?

✓ Yes

✗ No

Are the exact environment specifications used to generate a set of results included in the provenance records to link computational actions to particular environments?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

MUST A.1 Actionability

 1/2

Research object **MUST** contain sufficient instructions to reproduce all computational results.

Is a **README.md** or **Makefile** included with instructions for installation and usage?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Is there a clear starting point for users to start reproducing results (e.g., a main script, a workflow definition, or a container image)?

✓ Yes

✗ No

MUST P.1 Portability

 0/3

Procedures **MUST NOT** depend on undocumented host environment state.

Are relative paths used in scripts, avoiding hardcoded or system-specific paths like **C:\Users\...** or **/home/user/...** ?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Are all software dependencies included in the environment specification, rather than relying on pre-installed tools?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Have all assumptions about the host system's configuration been documented (e.g., specific OS versions, required system libraries, or environment variables)?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

MUST P.2 Portability

 0/1

Computational environments **MUST** be explicitly specified.

Is there a clear list of system requirements and dependencies documented in the README or environment specifications?

✓ Yes

✗ No

Reason for not meeting requirement...

MUST P.3 Portability **0/2**Environment definitions **MUST** be version controlled.

Are environment specifications (e.g., Dockerfiles, `pyproject.toml`, `package.json`) included in the version control system alongside code and data?

☒ Yes☐ No

0/250

Is there a process for updating environment specifications when dependencies change, and are these updates tracked in version control?

☒ Yes☐ No

0/250

MUST D.1 Distributability **0/3**All referenced modules and components **MUST** be persistently retrievable by others.

Are environment specifications (e.g., container digests, frozen package manifests) included to ensure others can attempt to exactly replicate the environment?

☒ Yes☐ No

0/250

Are environment specifications shared in a way that others can access and use them (e.g., published container images, archived environment files)?

☒ Yes☐ No

0/250

Is there documentation on how to obtain and use the environment specifications for reproduction?

☒ Yes☐ No

0/250

SHOULD T.2 Tracking **1/1**All components **SHOULD** be tracked using the same content-addressed version control system.

Is a common version control system (e.g., `Git`, `DVC`, `git-annex`, or `DataLad`) used across all components?

✓ Yes

✗ No

SHOULD **A.2** **Actionability**

💡 0/1

Procedures **SHOULD** be specified as executable specifications.

Is the workflow tested regularly to ensure instructions remain accurate?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

SHOULD **M.1** **Modularity**

💡 1/1

Components **SHOULD** be organized in a modular structure.

Are raw data, processed data, code, and environment definitions separated into distinct modules?

✓ Yes

✗ No

SHOULD **E.1** **Ephemerality**

💡 0/2

Computational results **SHOULD** be produced in ephemeral environments.

Is the pipeline tested in a fresh container, batch job, clean virtual machine, or cloud-based instance?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Does the workflow spawn disposable environments for each execution, allowing for isolation of runs?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

SHOULD **D.2** **Distributability**

💡 0/3

Environment specifications **SHOULD** support reproducible builds.

Are the exact environment specifications used to generate a specific set of results regularly tested to ensure they can be reconstituted as intended?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Are environment artifacts archived within the research object where possible (e.g., executable binaries, container images)?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Is there a process for updating environment specifications when dependencies change, and are these updates tracked in version control?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

MAY M.2 Modularity

💡 0/5

Components MAY be included directly or linked as subdatasets.

Are external datasets or software dependencies included as submodules or linked as submodules?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Is the modular structure documented to clarify how components relate and can be recombined?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Are modular boundaries defined in a way that allows independent updates without breaking the overall workflow?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Is there a clear mechanism for composing modules together (e.g., `git` submodules, or container orchestration)?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250

Is there documentation or tooling to support users in understanding and navigating the modular structure?

✓ Yes

✗ No

Reason for not meeting requirement...

0/250